



UNIVERSITE HASSAN II DE CASABLANCA
FACULTE DES SCIENCES BEN M'SICK

Documentation PFE - PDF 09/10

Workflow Complet

Du clic utilisateur au rendu final - Pas a pas

Realise par :

AKRAM BELMOUSSA - Backend & Integration NLP

ZAKARIA - Frontend Angular & UX/UI

NOUHAILA - Base de donnees & Contenu FAQ

MAY 2026

Sommaire

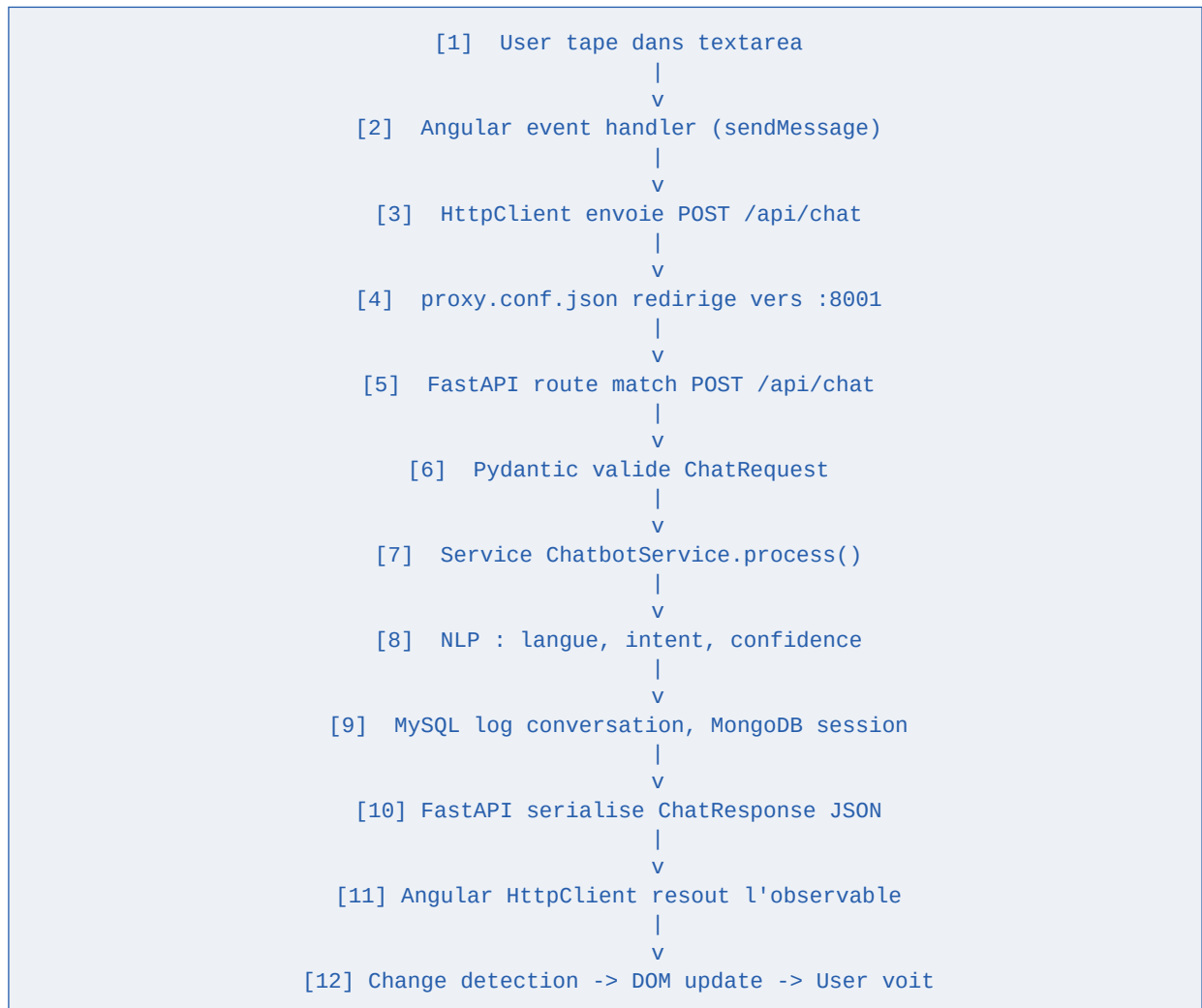
Chapitre 1 - Vue d'ensemble du workflow	3
Chapitre 2 - Etape 1 : L'utilisateur tape	6
Chapitre 3 - Etape 2 : Angular capture	9
Chapitre 4 - Etape 3 : HTTP request part	13
Chapitre 5 - Etape 4 : Proxy redirige	16
Chapitre 6 - Etape 5 : FastAPI recoit	19
Chapitre 7 - Etape 6 : Pydantic valide	23
Chapitre 8 - Etape 7 : Service traite	26
Chapitre 9 - Etape 8 : NLP analyse	30
Chapitre 10 - Etape 9 : SQL/Mongo persiste	34
Chapitre 11 - Etape 10 : Reponse JSON	37
Chapitre 12 - Etape 11 : Angular recoit	40
Chapitre 13 - Etape 12 : DOM se met a jour	43
Chapitre 14 - Cas particulier : LLM/RAG	46
Chapitre 15 - Conclusion	50

Chapitre 1 - Vue d'ensemble du workflow

1.1 Le scenario

On va suivre une requete **de bout en bout**. Scenario : un etudiant tape 'Quelles sont les filieres ?' dans le chatbot et voit la reponse.

1.2 Les 12 etapes



1.3 Latence totale

Etape	Temps approx	Pct
1-3 (Angular -> reseau)	~10 ms	5%
4 (proxy)	~2 ms	1%
5-6 (FastAPI + Pydantic)	~5 ms	2%
7-8 (NLP TF-IDF)	~30 ms	15%
9 (MySQL insert)	~15 ms	7%
9 (MongoDB upsert)	~10 ms	5%

10 (serialise JSON)	~3 ms	1%
11-12 (parse + DOM)	~5 ms	2%
Reseau aller-retour	~120 ms	60%
Total moyen	~200 ms	100%

Chapitre 2 - Etape 1 : L'utilisateur tape

2.1 Element HTML

```
<!-- chat-window.component.html -->
<textarea
  [(ngModel)]="currentMessage"
  (keydown)="onKeyDown($event)"
  placeholder="Pose ta question..."
  [disabled]="isTyping()"
  rows="1"
></textarea>

<button
  (click)="sendMessage()"
  [disabled]="!currentMessage.trim() || isTyping()"
>
  Envoyer
</button>
```

2.2 Ce qui se passe quand l'user tape une lettre

Browser detect keydown. Le navigateur capte l'evenement clavier.

Angular zone.js intercepte. Zone.js patche les APIs natives. Detecte le changement.

ngModel two-way binding. currentMessage est mis a jour.

Bouton se reactive. [disabled]='!currentMessage.trim()' devient false. Bouton activable.

Change detection. Angular re-evalue les bindings et met a jour le DOM si besoin.

2.3 Capture de la touche Entree

```
// chat-window.component.ts
onKeyDown(event: KeyboardEvent) {
  if (event.key === 'Enter' && !event.shiftKey) {
    event.preventDefault(); // empeche le retour ligne
    this.sendMessage();
  }
  // Shift+Enter = newline (laisse passer)
}
```

Chapitre 3 - Etape 2 : Angular capture

3.1 Methode sendMessage()

```
// chat-window.component.ts
sendMessage(): void {
  const text = this.currentMessage.trim();
  if (!text || this.isTyping()) return;

  // 1. Ajouter message user dans le state local
  this.messages.update(msgs => [...msgs, {
    sender: 'user',
    text,
    timestamp: new Date()
  }]);

  // 2. Vider la textarea
  this.currentMessage = '';

  // 3. Afficher l'indicateur 'is typing'
  this.isTyping.set(true);

  // 4. Choisir entre TF-IDF et LLM
  const useLLM = this.llmMode();
  const call$ = useLLM
    ? this.chat.sendMessageLLM(text, this.sessionId)
    : this.chat.sendMessage(text, this.sessionId);

  // 5. Souscrire
  call$.subscribe({
    next: (res) => this.handleResponse(res),
    error: (err) => this.handleError(err),
    complete: () => this.isTyping.set(false)
  });
}
```

3.2 Service ChatService

```
// services/chat.service.ts
@Injectable({ providedIn: 'root' })
export class ChatService {
  constructor(private http: HttpClient) {}

  sendMessage(message: string, sessionId: string): Observable<ChatResponse> {
    return this.http.post<ChatResponse>('/api/chat', {
      message,
      session_id: sessionId
    });
  }

  sendMessageLLM(message: string, sessionId: string): Observable<LLMResponse> {
    return this.http.post<LLMResponse>('/api/llm/chat', {
      message,
      session_id: sessionId,
      temperature: 0.6
    });
  }
}
```


Chapitre 4 - Etape 3 : HTTP request part

4.1 Sous le capot d'HttpClient

HttpClient retourne un Observable RxJS. Il ne fait **RIEN** tant qu'on ne souscrit pas. Au subscribe :

Construction de la requete. Methode, URL, headers, body JSON serialise.

Interceptors. Si configures (ex: ajouter Authorization). Pas en Phase 1.

Appel a fetch() ou XMLHttpRequest. Bas niveau. Le navigateur envoie sur le reseau.

DNS resolution. localhost = 127.0.0.1 (resolu instantanement).

TCP handshake. 3-way handshake avec le serveur. Quelques ms.

Envoi HTTP request. Le navigateur envoie les bytes.

4.2 La requete bytes finale

```
POST /api/chat HTTP/1.1
Host: localhost:4200
Connection: keep-alive
Content-Type: application/json
Content-Length: 71
Accept: application/json, text/plain, */*
User-Agent: Mozilla/5.0...
Origin: http://localhost:4200
Referer: http://localhost:4200/chatbot

{"message":"Quelles sont les filieres ?","session_id":"sess_abc123"}
```


Chapitre 5 - Etape 4 : Proxy redirige

5.1 Le serveur dev Angular intercepte

Le serveur Angular tourne sur :4200. Il a charge proxy.conf.json. Il inspecte l'URL de chaque requete.

5.2 Match du path

```
Requete entrante : POST http://localhost:4200/api/chat
```

```
proxy.conf.json :
```

```
{  
  '/api/chat': {  
    'target': 'http://localhost:8001',  
    'secure': false,  
    'changeOrigin': true  
  }  
}
```

```
Match ! /api/chat -> :8001
```

```
Nouvelle URL : POST http://localhost:8001/api/chat
```

5.3 Forwarding

```
Browser -> Angular Dev Server (:4200)  
Angular Dev Server -> Backend FastAPI (:8001)  
Backend repond -> Angular Dev Server (:4200)  
Angular Dev Server -> Browser
```

```
Le browser pense qu'il parle a :4200  
Pas de CORS car proxy = same origin
```

Chapitre 6 - Etape 5 : FastAPI recoit

6.1 Uvicorn ecoute :8001

Uvicorn = serveur ASGI qui execute FastAPI. Il ecoute sur le port 8001 et passe chaque requete a l'app FastAPI.

6.2 Routing FastAPI

```
# main.py
app = FastAPI(title='Chatbot Service')
app.include_router(chat_router, prefix='/api', tags=['chat'])

# routers/chat.py
@router.post('/chat', response_model=ChatResponse)
async def chat(
    req: ChatRequest,
    db: AsyncSession = Depends(get_db),
    mongo = Depends(get_mongo)
):
    ...
```

6.3 Le routeur fait

Resolution du chemin. POST /api/chat -> trouve la fonction `chat`.

Resolution des dependances. Depends(get_db) instancie une session SQLAlchemy. Depends(get_mongo) injecte le client Mongo.

Parse du body. FastAPI lit le body, le convertit en dict Python.

Validation Pydantic. Cree un ChatRequest a partir du dict. Si invalide -> 422.

Appel async de la fonction. await chat(req, db, mongo).

Chapitre 7 - Etape 6 : Pydantic valide

7.1 Le modele ChatRequest

```
from pydantic import BaseModel, Field

class ChatRequest(BaseModel):
    message: str = Field(..., min_length=1, max_length=2000)
    session_id: str | None = None
    user_id: int | None = None
    language: str | None = None
```

7.2 Que verifie Pydantic

- + **Types** : message doit etre une string
- + **Required** : message obligatoire (Field(...))
- + **Lengths** : $1 \leq \text{len}(\text{message}) \leq 2000$
- + **Optionnels** : session_id, user_id, language peuvent etre null
- + **Extra** : par default, ignore les champs non definis

7.3 Si validation echoue

```
Input invalide : { "message": "" } (vide)

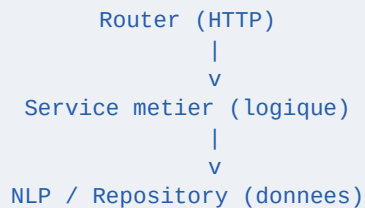
Reponse automatique :
HTTP 422 Unprocessable Entity
Content-Type: application/json

{
  "detail": [
    {
      "type": "string_too_short",
      "loc": ["body", "message"],
      "msg": "String should have at least 1 character",
      "input": "",
      "ctx": { "min_length": 1 }
    }
  ]
}
```

[★] Sans Pydantic, on devrait coder a la main : verifier que message existe, que c'est une string, que $\text{length} > 0$, etc. Pour chaque endpoint. Pydantic fait tout en 1 ligne.

Chapitre 8 - Etape 7 : Service traite

8.1 Architecture en couches



8.2 Le service ChatbotService

```

# services/chatbot.py
class ChatbotService:
    def __init__(self, classifieur, web_fetcher, persona):
        self.classifieur = classifieur
        self.web_fetcher = web_fetcher
        self.persona = persona

    async def process(
        self,
        message: str,
        session_id: str,
        db: AsyncSession,
        mongo,
        forced_language: str | None = None
    ) -> ChatResponse:
        # 1. Detecter ou recuperer la session
        if not session_id:
            session_id = generate_session_id()

        # 2. NLP : langue + intent
        result = self.classifieur.classify(message, forced_language)

        # 3. Detecter genre / nom
        gender = self.persona.detect_gender(message)
        name = self.persona.detect_name(message)

        # 4. Personnaliser la reponse
        response_text = self.persona.personalize(
            result.response, gender, name, result.language
        )

        # 5. Enrichir avec news si demande
        news_items = []
        if result.intent in ['actualites', 'evenements']:
            news_items = await self.web_fetcher.fetch_news(
                lang=result.language
            )

        # 6. Persister en BDD
        conversation_id = await self._save_conversation(
            db, session_id, message, response_text, result
        )
        await self._save_session(mongo, session_id, ...)
  
```

```
# 7. Retourner reponse
return ChatResponse(
    response=response_text,
    intent=result.intent,
    confidence=result.confidence,
    session_id=session_id,
    language=result.language,
    conversation_id=conversation_id,
    news_items=news_items,
    suggestions=result.suggestions
)
```

Chapitre 9 - Etape 8 : NLP analyse

9.1 Detection de langue

```
# language_detector.py
def detect(self, text: str) -> tuple[str, float]:
    # 1. Hint script arabe -> darija
    if re.search(r'[\u0600-\u06FF]', text):
        return ('darija', 1.0)

    # 2. Hint numerique darija (3, 7, 9)
    if re.search(r'\b\w*[379]\w*\b', text):
        return ('darija', 0.9)

    # 3. Compte mots-cles FR vs EN vs Darija
    text_lower = text.lower()
    scores = {
        'fr': sum(1 for k in FR_KEYWORDS if k in text_lower),
        'en': sum(1 for k in EN_KEYWORDS if k in text_lower),
        'darija': sum(1 for k in DARIJA_KEYWORDS if k in text_lower),
    }

    # 4. Le max gagne
    best = max(scores, key=scores.get)
    total = sum(scores.values()) or 1
    confidence = scores[best] / total
    return (best, confidence)
```

9.2 Classification d'intent

```
# classifieur.py
def classify(self, text: str, forced_lang=None) -> ClassResult:
    # 1. Detecter langue
    lang, lang_conf = self.detect(text)
    if forced_lang:
        lang = forced_lang

    # 2. Choisir vectorizer + matrix pour cette langue
    vectorizer = self.vectorizers[lang]
    pattern_matrix = self.matrices[lang]
    intents = self.intents[lang]

    # 3. Vectoriser la question user
    user_vec = vectorizer.transform([text.lower()])

    # 4. Cosine similarity vs TOUS les patterns
    similarities = cosine_similarity(user_vec, pattern_matrix)[0]

    # 5. Top 3 candidats
    top_idx = similarities.argsort()[::-1][:3]

    # 6. Best intent + sa confidence
    best_intent = intents[top_idx[0]]
    confidence = similarities[top_idx[0]]

    # 7. Choisir reponse aleatoire de cet intent
```

```
response = random.choice(best_intent.responses[lang])

return ClassResult(
    intent=best_intent.tag,
    response=response,
    confidence=confidence,
    language=lang,
    top_candidates=[
        (intents[i].tag, similarities[i]) for i in top_idx
    ]
)
```

Chapitre 10 - Etape 9 : SQL/Mongo persiste

10.1 Insert dans MySQL conversations

```

async def _save_conversation(
    self, db, session_id, message, response, result
) -> int:
    conv = Conversation(
        session_id=session_id,
        user_message=message,
        bot_response=response,
        intent_detected=result.intent,
        confidence_score=result.confidence,
        language=result.language,
        response_time_ms=result.response_time_ms
    )
    db.add(conv)
    await db.commit()
    await db.refresh(conv)
    return conv.id

```

10.2 Upsert dans MongoDB sessions

```

async def _save_session(
    self, mongo, session_id, message, response, result
):
    sessions = mongo.get_collection('sessions')

    # Append au tableau messages
    await sessions.update_one(
        {'session_id': session_id},
        {
            '$setOnInsert': {
                'session_id': session_id,
                'created_at': datetime.utcnow(),
                'language': result.language
            },
            '$set': {
                'updated_at': datetime.utcnow(),
                'last_intent': result.intent
            },
            '$push': {
                'messages': {
                    'sender': 'user',
                    'text': message,
                    'timestamp': datetime.utcnow()
                }
            },
            '$inc': { 'message_count': 1 }
        },
        upsert=True # cree si pas existe
    )

```

10.3 Sous le capot SQL


```
INSERT INTO conversations (  
    session_id, user_message, bot_response,  
    intent_detected, confidence_score, language,  
    response_time_ms, created_at  
) VALUES (  
    ?, ?, ?, ?, ?, ?, ?, NOW()  
);  
  
-- Parametres (echappes par SQLAlchemy)  
-- ?1 = 'sess_abc123'  
-- ?2 = 'Quelles sont les filieres ?'  
-- ?3 = 'La FSBM propose 7 licences...'  
-- ?4 = 'filieres'  
-- ?5 = 1.0  
-- ?6 = 'fr'  
-- ?7 = 152
```

Chapitre 11 - Etape 10 : Reponse JSON

11.1 ChatResponse Pydantic

```
class ChatResponse(BaseModel):
    response: str
    intent: str
    confidence: float
    conversation_id: int
    session_id: str
    language: str
    language_confidence: float
    top_candidates: list[dict]
    suggestions: list[str]
    news_items: list[dict] = []
    response_time_ms: int
```

11.2 Serialisation auto FastAPI

```
# Le router retourne un ChatResponse
return ChatResponse(
    response='La FSBM propose 7 licences...',
    intent='filieres',
    confidence=1.0,
    session_id='sess_abc123',
    ...
)

# FastAPI fait automatiquement :
# 1. response.model_dump() -> dict
# 2. json.dumps(dict) -> string JSON
# 3. HTTP 200 + Content-Type: application/json
# 4. Envoie au client
```

11.3 Le JSON final envoye

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 487
Date: Wed, 28 May 2026 13:24:35 GMT
Server: uvicorn

{
  "response": "La FSBM propose 7 licences et 18 masters...",
  "intent": "filieres",
  "confidence": 1.0,
  "conversation_id": 42,
  "session_id": "sess_abc123",
  "language": "fr",
  "language_confidence": 1.0,
  "top_candidates": [
    {"intent": "filieres", "score": 1.0},
    {"intent": "masters", "score": 0.65},
    {"intent": "licences", "score": 0.45}
  ],
  "suggestions": ["Master IADS", "Licence SMI"],
```

```
"news_items": [],  
"response_time_ms": 152  
}
```

Chapitre 12 - Etape 11 : Angular recoit

12.1 Le browser recoit les bytes

TCP recoit les bytes. Reassemble les paquets en ordre.

HTTP parse headers + body. Sapare headers et body.

Decodage UTF-8. Les bytes deviennent une string.

JSON.parse. La string devient un objet JavaScript.

Observable resout. RxJS emit le 'next' avec la reponse.

Subscribe handler appelle. Notre subscribe({ next: ... }) est invoque.

12.2 handleResponse

```
handleResponse(res: ChatResponse): void {  
  // 1. Ajouter message bot dans le state  
  this.messages.update(msgs => [...msgs, {  
    sender: 'bot',  
    text: res.response,  
    timestamp: new Date(),  
    intent: res.intent,  
    confidence: res.confidence,  
    newsItems: res.news_items  
  }]);  
  
  // 2. Mettre a jour suggestions  
  this.suggestions.set(res.suggestions);  
  
  // 3. Memoriser conv_id pour feedback  
  this.lastConversationId = res.conversation_id;  
  
  // 4. Cacher l'indicateur 'typing'  
  this.isTyping.set(false);  
  
  // 5. Scroller vers le bas  
  setTimeout(() => this.scrollToBottom(), 50);  
}
```

Chapitre 13 - Etape 12 : DOM se met a jour

13.1 Change Detection

Angular utilise **OnPush + Signals** dans notre projet. Quand on appelle `this.messages.update(...)`, le signal notifie tous les abonnes.

13.2 Le template re-render

```
<!-- chat-window.component.html -->
<div class="messages">
  @for (msg of messages(); track $index) {
    <div [class]="message ' ' + msg.sender">
      <div class="avatar">
        @if (msg.sender === 'bot') { Bot } @else { User }
      </div>
      <div class="text">{{ msg.text }}</div>

      @if (msg.newsItems?.length) {
        <div class="news-items">
          @for (n of msg.newsItems; track n.url) {
            <a [href]="n.url" target="_blank">{{ n.title }}</a>
          }
        </div>
      }
    </div>
  }
</div>
```

13.3 Le rendu visuel

Angular detecte signal update. Marque le composant comme 'dirty'.

Angular re-execute le template. Genere une nouvelle representation virtuelle (VDOM-like).

Diff avec DOM actuel. Trouve les differences.

Patch le DOM. Applique seulement les diffs. Ajoute le nouveau message.

Browser repaint. Le navigateur re-dessine la zone modifiee.

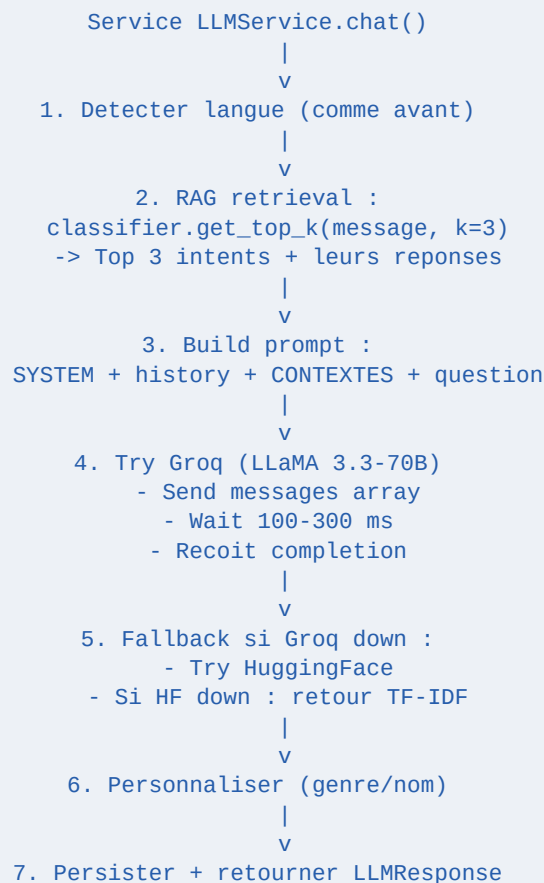
L'user voit le nouveau message. Tout cela en <16 ms (60 FPS).

Chapitre 14 - Cas particulier : LLM/RAG

14.1 Quand l'utilisateur active LLM mode

Le workflow change a partir de l'étape 7. Au lieu du ChatbotService, on appelle LLMService.

14.2 Nouveau workflow LLM



14.3 Le prompt envoye a Groq

```

messages = [
  {
    'role': 'system',
    'content': """"Tu es l'assistant officiel de la Faculte des Sciences
Ben M'Sick (FSBM), Universite Hassan II Casablanca.
Tu reponds en francais de maniere claire et bienveillante.
Tu utilises EXCLUSIVEMENT les informations du contexte ci-dessous.
Si tu ne sais pas, dis-le.""
  },
  {
    'role': 'system',
    'content': """"## CONTEXTES PERTINENTS

[CONTEXTE 1] (intent: master_iads, score: 0.91)
Le Master IADS - IA et Data Science - dure 2 ans, capacite 25 places.
Public vise : titulaires de licence informatique ou maths.
Modules : Machine Learning, Deep Learning, NLP, Computer Vision...
  
```

Coordinateur : Pr. Mehdi FILALI (m.filali@fsbm.ac.ma).

[CONTEXTE 2] (intent: masters, score: 0.45)

La FSBM propose 18 masters repartis en 4 departements..."

```
},
# Historique conversation
{'role': 'user', 'content': 'Salut'},
{'role': 'assistant', 'content': 'Bonjour ! Comment puis-je aider ?'},
# Question actuelle
{'role': 'user', 'content': 'Conditions pour le master IADS ?'}
]
```

Total : ~800 tokens (input)

Reponse attendue : ~200 tokens

14.4 Reponse Groq

POST https://api.groq.com/openai/v1/chat/completions

Authorization: Bearer gsk_XXX

Content-Type: application/json

```
{
  "model": "llama-3.3-70b-versatile",
  "messages": [...], // comme ci-dessus,
  "temperature": 0.6,
  "max_tokens": 800
}
```

Reponse :

```
{
  "choices": [{
    "message": {
      "role": "assistant",
      "content": "Pour le master IADS, vous devez avoir une licence..."
    }
  }],
  "usage": {
    "prompt_tokens": 812,
    "completion_tokens": 187,
    "total_tokens": 999
  }
}
```

[✓] LLaMA 3.3-70B genere sa reponse en se basant SUR LES CONTEXTES (RAG). Sans RAG, le modele invente. Avec RAG, il reformule de maniere naturelle des infos fiables.

Chapitre 15 - Conclusion

15.1 Le voyage complet

On a suivi une requete de bout en bout. Sur ~200 ms :

- + 60% en transit reseau (TCP + HTTP)
- + 15% en NLP TF-IDF (vectorisation + cosine)
- + 12% en persistance MySQL + MongoDB
- + 5% en frontend (Angular + DOM)
- + 8% en autres (proxy, FastAPI, Pydantic)

15.2 Couches traversees

Couche	Role
Browser DOM	Capture input + render output
Angular	Etat, binding, composants
HttpClient + RxJS	Requete reseau, observable
Proxy Dev	CORS bypass, routing
Uvicorn	Serveur ASGI
FastAPI	Router, dependency injection
Pydantic	Validation des inputs
Service metier	Orchestration
NLP TF-IDF	Classification d'intent
LLM optionnel	Generation naturelle
SQLAlchemy	ORM async MySQL
Motor	Client async MongoDB

15.3 Cle de comprehension

Chaque couche a une responsabilite claire. Angular ne sait rien de MySQL. FastAPI ne sait rien d'Angular. NLP ne sait rien du reseau. Ce decouplage permet de modifier une couche sans impacter les autres.

15.4 Pour aller plus loin

- PDF 02 - frontend Angular en detail
- PDF 03 - backend FastAPI

- PDF 06 - NLP et IA
- PDF 07 - APIs et endpoints